Game Data Capture Spec

Version 2.1 - 07/24/2026

1 Canonical Format 2
1.1 Vendor Upload Format 2
1.1.1 Upload Path for Data Delivery 2
1.1.2 Session File Structure 2
1.1.2.1 Session.json File Requirements 3
1.1.2.2 Frames.csv File Requirements 10
1.1.2.3 World Coordinate System 10
1.1.2.4 Camera-to-World (C2W) Matrix 11
1.1.2.5 Camera Intrinsics 11
1.1.5.6 Control / Input Data 12
1.1.3 Complete Frames.csv Column Summary 13
1.4 Session.rrd & rrd_creation.py 14
1.5 Vendor Validation Checks 14
1.5.1 session.json 14
1.5.2 frames.csv — Structure 15
1.5.3 frames.csv — C2W Matrix 15
1.5.4 frames.csv — Camera Intrinsics 15
1.5.5 frames.csv — Control / Input Data 16
1.5.6 video.mp4 16
2 Appendix: Odyssey Internal Coordinate Conversion Reference 16
2.1 3-Letter Axis Notation 16
2.2 Converting Camera Columns to OpenGL Convention 17
2.3 (GTA V) Conversion Notes 17
2.4 (RDR2) Conversion Notes 17
2.5 Cyberpunk Notes 18
2.6 Hogwarts Legacy Notes 18

Change log:

- Expanded `session.json` schema and requirements
  - Adds `input_mouse_convention` to detail semantic mouse understanding
- Removed `key_binding.json` file inclusion requirement
  - In its place, use per-frame `input_keys` and `input_actions` in `frames.csv`
  - The requirement caused confusion when 1 key was used contextually for >1 action (e.g. “E” can be enter OR exit vehicle depending on players’ current position).
- Clarified `input_data` requirements in `frames.csv`

- When a vendor provides a subset of specification, clarified that all `frames.csv` column headers must be included and their values should be `null`

1 Canonical Format

This document describes the standard Game Data format that Odyssey will use as a normalised canonical source of data for downstream processing.

Every vendor delivery must include the full `frames.csv` schema, even if vendor is only providing a subset of the data specified in this document (e.g. only .mp4 video files and control input – but no camera metadata); Retain each of the spec's column headers and populate the extra headers with null values.

1.1 Vendor Upload Format

Vendors should deliver their data to the vendor-specific cloudflare r2 bucket provided by Odyssey using the access credentials provided by Odyssey.

1.1.1 Upload Path for Data Delivery

```
<vendor>/<mm-dd-yyyy>/<game>/<session-id>/..
```

⚠ `<vendor>` is your root bucket path by Odyssey.
⚠ `<mm-dd-yyyy>` must be the UTC date of data delivery (upload).

Cloudflare R2 uses a flat storage structure – to upload your data into specific nested file paths, see: https://developers.cloudflare.com/r2/objects/

1.1.2 Session File Structure

```
<vendor>/<mm-dd-yyyy>/<game>/<session-id>/...
  session.json       # metadata sidecar
  frames.csv         # per-frame camera + input data
  video.mp4          # screen recording
  session.rrd        # the .rrd file you've created that visualizes all frame-synced data
  rrd_creation.py    # the script used to create your provided session's .rrd file
```

1.1.2.1 Session.json File Requirements

Access the schema in the code block below as a .json file here.

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "session_schema.json",
  "title": "Session",
  "description": "Metadata schema for a recorded gameplay session.",
  "type": "object",
  "required": [
    "vendor_name",
    "game_title",
    "session_id",
    "created_at_utc",
    "ended_at_utc",
    "duration_ms",
    "duration_seconds",
    "fps",
    "frame_count",
    "record_width_px",
    "record_height_px",
    "screen_width_px",
    "screen_height_px",
    "localization",
    "platform",
    "input_mouse_convention"
  ],
  "additionalProperties": true,
  "$defs": {
    "maps_to_value": {
      "type": "string",
      "oneOf": [
        {
          "const": "camera_look_velocity",
          "description": "Camera Rotation (Speed-Based): Raw mouse deltas turn the
character's head or rotate the 3D viewport. Faster movement equals faster
rotation. Cursor is usually locked and hidden. Examples: Call of Duty, Halo, Elden
Ring."
        },
        {
          "const": "camera_pan_velocity",
          "description": "Camera Translation (Speed-Based): Moving the mouse or
dragging across the screen slides the entire camera anchor linearly across a flat
2D plane or map. Examples: StarCraft, Age of Empires, Cities: Skylines."
        },
        {
          "const": "camera_pan_position",
          "description": "Camera Direction (Absolute Position-Based): The absolute
screen coordinates of the mouse dictate the camera's movement or tilt. Common in
```

```
edge-scrolling or virtual joystick flight setups. Examples: Microsoft Flight
Simulator, strategy edge-panning."
        },
        {
          "const": "cursor_position",
          "description": "2D Interface Pointer (Absolute Position-Based): The
mouse controls a visible, free-roaming screen cursor used to click UI buttons,
manage inventories, or select items. Examples: Monkey Island, League of Legends
menus."
        },
        {
          "const": "vehicle_steering",
          "description": "Mechanical Control Mapping: Mouse movement mimics a
physical steering wheel, flight yoke, or handlebars to turn a vehicle rather than
just controlling a camera viewpoint. Examples: War Thunder aircraft, GTA driving."
        },
        {
          "const": "none",
          "description": "No Mouse Interaction: Mouse input is entirely
disregarded by the game engine during primary gameplay. Examples: Fighting games
(Mortal Kombat), pure gamepad/keyboard platformers."
        },
        {
          "const": "other",
          "description": "Custom/Unique Interaction: The mouse drives an entirely
custom mechanic not covered by the standard options. Note: Requires
'maps_to_other' to be populated. Examples: Mouse gestures for spellcasting,
lockpicking mini-games."
        }
      ]
    }
  },
  "properties": {
    "vendor_name": {
      "type": "string",
      "description": "Name of the vendor who captured and delivered the data."
    },
    "game_title": {
      "type": "string",
      "description": "Title of the game being played."
    },
    "session_id": {
      "type": "string",
      "description": "Unique identifier for the session."
    },
    "created_at_utc": {
      "type": "string",
      "format": "date-time",
      "pattern":
"^\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}:\\d{2}(\\.\\d+)?(Z|[+-]\\d{2}:\\d{2})$",
```

```
      "description": "UTC timestamp when the session recording began
(timezone-aware ISO 8601, e.g. '2026-06-15T10:00:00Z')."
    },
    "ended_at_utc": {
      "type": "string",
      "format": "date-time",
      "pattern":
"^\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}:\\d{2}(\\.\\d+)?(Z|[+-]\\d{2}:\\d{2})$",
      "description": "UTC timestamp when the session recording ended
(timezone-aware ISO 8601, e.g. '2026-06-15T11:00:00Z')."
    },
    "duration_ms": {
      "type": "number",
      "minimum": 0,
      "description": "Total duration of the recording in milliseconds."
    },
    "duration_seconds": {
      "type": "number",
      "exclusiveMinimum": 0,
      "description": "Total duration of the recording in seconds."
    },
    "fps": {
      "type": "number",
      "exclusiveMinimum": 0,
      "description": "Frames per second of the recording."
    },
    "frame_count": {
      "type": "integer",
      "minimum": 1,
      "description": "Total number of frames in the recording."
    },
    "record_width_px": {
      "type": "integer",
      "minimum": 1,
      "description": "Width of the recorded video in pixels."
    },
    "record_height_px": {
      "type": "integer",
      "minimum": 1,
      "description": "Height of the recorded video in pixels."
    },
    "screen_width_px": {
      "type": "integer",
      "minimum": 1,
      "description": "Width of the source screen/display in pixels."
    },
    "screen_height_px": {
      "type": "integer",
      "minimum": 1,
      "description": "Height of the source screen/display in pixels."
    },
```

```
    "localization": {
      "type": "string",
      "pattern": "^[a-z]{2,3}(-[A-Z]{2,3})?$",
      "description": "BCP 47 language tag representing the locale of the session
(e.g. 'en-US', 'ja-JP')."
    },
    "platform": {
      "type": "string",
      "enum": [
        "Xbox",
        "PC",
        "Switch",
        "PlayStation",
        "Mobile-iOS",
        "Mobile-Android",
        "Steam Deck"
      ],
      "description": "The gaming platform on which the session was recorded."
    },
    "input_mouse_convention": {
      "type": "object",
      "required": [
        "maps_to",
        "dx_positive",
        "dx_negative",
        "dy_positive",
        "dy_negative"
      ],
      "additionalProperties": false,
      "properties": {
        "maps_to": {
          "$ref": "#/$defs/maps_to_value",
          "description": "Indicates the primary system or camera quantity driven
by raw mouse inputs during the main gameplay mode of the session. When behavior
varies within the session, this is the predominant mapping; deviations are
declared in input_mouse_convention_segments."
        },
        "maps_to_other": {
          "type": "string",
          "description": "Mandatory human-readable string clarifying the specific
behavior if 'maps_to' is set to 'other'."
        },
        "dx_positive": {
          "type": "string",
          "enum": [
            "right",
            "left",
            "not_applicable"
          ],
```

```
          "description": "Camera/view response to a positive input_mouse_dx delta
(moving physical mouse to player's right). Reflects actual in-session settings as
observed from the player's perspective."
        },
        "dx_negative": {
          "type": "string",
          "enum": [
            "right",
            "left",
            "not_applicable"
          ],
          "description": "Camera/view response to a negative input_mouse_dx delta
(moving physical mouse to player's left). Reflects actual in-session settings as
observed from the player's perspective."
        },
        "dy_positive": {
          "type": "string",
          "enum": [
            "down",
            "up",
            "not_applicable"
          ],
          "description": "Camera/view response to positive input_mouse_dy --
moving the physical mouse down (towards player's body). Captures whether invert-Y
was enabled."
        },
        "dy_negative": {
          "type": "string",
          "enum": [
            "down",
            "up",
            "not_applicable"
          ],
          "description": "Camera/view response to negative input_mouse_dy --
moving the physical mouse up (away from player's body). Captures whether invert-Y
was enabled."
        }
      },
      "allOf": [
        {
          "if": {
            "properties": {
              "maps_to": {
                "enum": [
                  "camera_look_velocity",
                  "camera_pan_velocity",
                  "camera_pan_position"
                ]
              }
            },
            "required": [
```

```
          "maps_to"
        ]
      },
      "then": {
        "properties": {
          "dx_positive": {
            "enum": [
              "right",
              "left"
            ]
          },
          "dx_negative": {
            "enum": [
              "right",
              "left"
            ]
          },
          "dy_positive": {
            "enum": [
              "down",
              "up"
            ]
          },
          "dy_negative": {
            "enum": [
              "down",
              "up"
            ]
          }
        }
      },
      "else": {
        "properties": {
          "dx_positive": {
            "const": "not_applicable"
          },
          "dx_negative": {
            "const": "not_applicable"
          },
          "dy_positive": {
            "const": "not_applicable"
          },
          "dy_negative": {
            "const": "not_applicable"
          }
        }
      }
    },
    {
      "if": {
        "properties": {
```

```
              "maps_to": {
                "const": "other"
              }
            },
            "required": [
              "maps_to"
            ]
          },
          "then": {
            "required": [
              "maps_to_other"
            ]
          }
        }
      ],
      "description": "Declares what the raw mouse deltas control (maps_to) and,
for active camera control mappings, the directional/inversion convention. The
predominant mapping for the session; see input_mouse_convention_segments for
within-session deviations."
    },
    "input_mouse_convention_segments": {
      "type": "array",
      "minItems": 1,
      "description": "Optional. Declares frame ranges where the
mouse-to-camera/game behavior deviates from the session-level
input_mouse_convention.maps_to (e.g. GTA: on-foot camera_look_velocity session
with a vehicle_steering segment while driving). Frames not covered by any segment
are governed by the session-level mapping. Segments must not overlap. Frame
indices are 0-based and align with frames.csv row order; start_frame is inclusive,
end_frame is exclusive.",
      "items": {
        "type": "object",
        "required": [
          "start_frame",
          "end_frame",
          "maps_to"
        ],
        "additionalProperties": false,
        "properties": {
          "start_frame": {
            "type": "integer",
            "minimum": 0,
            "description": "First frame of the segment (inclusive, 0-based,
aligned with frames.csv)."
          },
          "end_frame": {
            "type": "integer",
            "minimum": 1,
            "description": "Frame after the last frame of the segment (exclusive,
0-based, aligned with frames.csv)."
          },
```

```
          "maps_to": {
            "$ref": "#/$defs/maps_to_value",
            "description": "Mouse-to-camera/game behavior in effect during this
segment."
          },
          "maps_to_other": {
            "type": "string",
            "description": "Mandatory human-readable string clarifying the
specific behavior if this segment's 'maps_to' is set to 'other'."
          }
        },
        "if": {
          "properties": {
            "maps_to": {
              "const": "other"
            }
          },
          "required": [
            "maps_to"
          ]
        },
        "then": {
          "required": [
            "maps_to_other"
          ]
        }
      }
    }
  }
}
```

1.1.2.2 Frames.csv File Requirements

`Frames.csv` must follow the conventions and column names described within this section.

1.1.2.3 World Coordinate System

Right-handed Z-up (`RFU`)

```
      Z (up)
      |
      |   Y (forward/north)
      |  /
      | /
      +————————> X (right/east)
```

- X = right / east
- Y = forward / north

- Z = up
- Units: game-world units (typically metres)
- Handedness: right-handed (`cross(X, Y) = Z`)

1.1.2.4 Camera-to-World (C2W) Matrix

We will use Camera-to-World matrix to store the camera information as this is generally the form that is most easily captured from game data. Downstream, this can be converted to whatever the the training pipeline requires eg Poses that are world-to-camera/RDF

The C2W matrix will be using row-major, is 4×4 homogeneous, and RUB coordinate systems (x=right, y=up, z=backward)

$$\begin{bmatrix} \text{c2w_m00} & \text{c2w_m01} & \text{c2w_m02} & \text{c2w_m03} \\ \text{c2w_m10} & \text{c2w_m11} & \text{c2w_m12} & \text{c2w_m13} \\ \text{c2w_m20} & \text{c2w_m21} & \text{c2w_m22} & \text{c2w_m23} \\ \text{c2w_m30} & \text{c2w_m31} & \text{c2w_m32} & \text{c2w_m33} \end{bmatrix} = \begin{bmatrix} R_x & U_x & B_x & T_x \\ R_y & U_y & B_y & T_y \\ R_i & U_i & B_i & T_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Column	Name	Meaning
0 (R)	Right	Camera right axis in world. Horizontal when no roll ($R_i = 0$).
1 (U)	Up	Camera up axis in the world. Points roughly toward +Z when the camera is upright.
2 (B)	Backward	Opposite of the look direction. The camera looks along −column 2.
3 (T)	Translation	Camera position in world coordinates.

Constraints:

- Bottom row: `(0, 0, 0, 1)`
- Rotation determinant: `+1` (right-handed)
- Row-major storage: `c2w_mRC` means row R, column C

1.1.2.5 Camera Intrinsics

The camera intrinsics will be represented by a perspective matrix. In the parquet and CSV, we just require fx, fy, cx, cy though we'll have an extra optional field for other standard coefficients.

If providing camera metadata, it must be provided dynamically for every frame – some game titles have >1 camera / field of view that they switch between.

Generally , the perspective matrix looks like this:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_Y & c_Y \\ 0 & 0 & 1 \end{bmatrix}$$

There is also a **camera_model** field. This gives us the camera model that is used. Most game captures will use PINHOLE but the others are provided for future extension.

Model	Required Columns
PINHOLE	camera_fx, camera_fy, camera_cx, camera_cy
OPENCV	Above + camera_radial_k1, k2, camera_tangential_p1, p2
OPENCV_FISHEYE	Above + camera_radial_k1 .. camera_radial_k4

Vendors can compute pinhole intrinsics from FOV + resolution:

```
f_Y = height / (2 × tan(fov_vertical / 2))
f_x = f_Y                                    [for square pixels]
c_x = width / 2
c_Y = height / 2
```

1.1.5.6 Control / Input Data

Input events are stored at the video frame rate (one row per frame, same as camera data).

All data that is captured at a higher frame rate than the video frame rate should be accumulated.

Column	Type	Format
input_keys	string optional[null]	Per-frame, pipe-delimited active keys. Null if no per-frame active keys. E.g. W|Shift
input_actions	string optional[null]	Per-frame, pipe-delimited, active semantic in-game actions. Null if no per-frame active keys. E.g. “Jump” “Forward | Jump” “Enter Vehicle” “Shoot”
input_mouse_buttons	string	Pipe-delimited active mouse buttons. E.g. Left|Right
input_mouse_dx	float	Mouse movement delta X (unit not specified)

Column	Type	Format
		“0.0” if no delta
input_mouse_dy	float	Mouse movement delta Y (unit not specified) “0.0” if no delta

1.1.3 Complete Frames.csv Column Summary

Sessions may have camera data, input data, or both. Camera columns are null for sessions without camera capture — Odyssey’s viewer and data tables hide them automatically.

Column(s)	Type	Required	Description
frame_id	int	Yes	Zero-based frame index
timestamp_ms	int	Yes	Milliseconds from start of capture
C2W_M00 C2W_M01 C2W_M02 C2W_M03 C2W_M10 C2W_M11 C2W_M12 C2W_M13 C2W_M20 C2W_M21 C2W_M22 C2W_M23 C2W_M30 C2W_M31 C2W_M32 C2W_M33	float	When providing camera data	C2W 4×4 matrix (see above). Null if no camera.
camera_model	string	When providing camera data	COLMAP camera model name. Null no camera.
camera_fx	float	When providing camera data	Focal length X in pixels (`fx = fy` for square pixels)
camera_fy	float	When providing camera data	Focal length Y in pixels
camera_cx	float	When providing camera data	Principal point X in pixels (`width / 2`)
camera_cy	float	When providing camera data	Principal point Y in pixels (`height / 2`)
camera_radial_k1 camera_radial_k2 camera_radial_k3 camera_radial_k4 camera_radial_k5 camera_radial_k6	float	No	Radial distortion coefficients

Column(s)	Type	Required	Description
`camera_tangential_p1` `camera_tangential_p2`	float	No	Tangential distortion coefficients
`input_keys`	string	Yes	Per-frame, pipe-delimited active keys. Null if no per-frame active keys. E.g. `W|Shift`
`input_actions`	string	Yes	Per-frame, pipe-delimited, active semantic in-game actions. Null if no per-frame active keys. E.g. "`Jump`" "`Forward|Jump`" "`Enter Vehicle`" "`Shoot`"
`input_mouse_buttons`	string	Yes	Pipe-delimited active mouse buttons. E.g. `Left|Right`
`input_mouse_dx`	float	Yes	Mouse movement delta X (unit not specified) "0.0" if no delta
`input_mouse_dy`	float	Yes	Mouse movement delta Y (unit not specified) "0.0" if no delta

Note on intrinsics: For games with square pixels (most), `fx = fy`. Derive from projection matrix: `fy = proj[1][1] × height / 2`, `fx = fy`. Do NOT derive `fx` independently from `proj[0][0]` — that encodes the viewport aspect, not a different focal length.

1.4 Session.rrd & rrd_creation.py

Vendor provided `.rrd` files and scripts ensures that Odyssey and the vendor have a common baseline for comparative QA.

For quality assurance, a random 15% of every delivered batch must include:

1. `.rrd` file to visualize video frames alongside frame-synced metadata in https://rerun.io/
2. The script/code used to generate the `.rrd` files you provide

1.5 Vendor Validation Checks

Checks vendors should perform to verify their data conforms to Odyssey's canonical spec:

1.5.1 session.json

Check	Expected	What It Means
`All required fields present`	No missing keys	session.json validates against session_schema.json
`ended_at_utc > created_at_utc`	Positive delta	Session end is after session start
`duration_ms / 1000 ≈ duration_seconds`	Within 1 s	Both duration fields agree
`duration_ms ≈ ended_at_utc − created_at_utc`	Within 1 s	Duration consistent with timestamps

`frame_count ≈ fps × duration_seconds`	Within 2 frames	Frame count consistent with fps and duration
`Timestamps include timezone offset`	Z or ±HH:MM suffix	Timezone-aware ISO 8601 — naive datetimes are rejected

1.5.2 frames.csv — Structure

Check	Expected	What It Means
`Row count == frame_count`	Exact match	One row per video frame, no missing or extra rows
`All spec column headers present`	No missing headers	Columns not captured should be present with null values, not omitted
`frame_id is zero-based and sequential`	0, 1, 2 … N−1, no gaps	No dropped or duplicated frame indices
`timestamp_ms is monotonically increasing`	Each value > previous	No out-of-order or duplicate timestamps
`timestamp_ms[0] ≈ 0`	Near zero	Timestamps are relative to session start, not wall time
`timestamp_ms[-1] ≈ duration_ms`	Within one frame interval	Last frame timestamp consistent with session duration
`Frame interval ≈ 1000 / fps ms`	Consistent spacing	Fixed-rate capture — no irregular gaps

1.5.3 frames.csv — C2W Matrix

Check	Expected	What It Means
`det(R) = +1`	Exactly +1	Right-handed rotation
`c2w_m20 ≈ 0`	Near zero for level cameras	Right axis is horizontal (no roll)
`c2w_m21 > 0`	Positive for upright cameras	Up axis has a positive Z component
`c2w_m23`	Reasonable height value	Position Z is height above ground
`c2w_m30, m31, m32 = 0`	Exactly zero	Standard affine matrix
$R \times R^T = I$	Identity (within float precision)	Rotation is orthonormal

1.5.4 frames.csv — Camera Intrinsics

Check	Expected	What It Means
`camera_model is PINHOLE, OPENCV, or OPENCV_FISHEYE`	Valid enum value	No freeform strings
`camera_fx, fy, cx, cy all non-null when camera_model is set`	Non-null	Intrinsics fully populated for every frame with a camera model
`camera_cx ≈ record_width_px / 2`	Within a few pixels	Principal point is near image centre
`camera_cy ≈ record_height_px / 2`	Within a few pixels	Principal point is near image centre
`camera_fx ≈ camera_fy`	Equal for square pixels	Derived from projection matrix correctly — not independently from proj[0][0]

`Camera columns non-null for every frame`	No sparse nulls mid-session	Intrinsics must be provided per-frame, including during FOV changes

1.5.5 frames.csv — Control / Input Data

Check	Expected	What It Means
`Frames with active input_keys have non-null input_actions`	No keys without actions	Every key press must map to a semantic action
`Frames with no input have null input_keys and input_actions`	Null, not empty string	Distinguish "no input" from "input not captured"
`input_mouse_dx and input_mouse_dy are "0.0" when no movement`	"0.0", not null	These fields use 0.0 sentinel, not null, for no-movement frames
`Delimiter is | (pipe), not comma or space`	Pipe-separated	e.g. W|+|Shift, not W,+,Shift

1.5.6 video.mp4

Check	Expected	What It Means
`Video resolution == record_width_px × record_height_px`	Exact match	session.json must reflect the actual encoded resolution
`Video frame count == frames.csv row count`	Exact match	frames.csv row count and video frame count must agree
`Video duration ≈ duration_seconds`	Within 1 s	Encoded duration consistent with session metadata

2 Appendix: Odyssey Internal Coordinate Conversion Reference

For internal use when writing preprocessors for legacy vendor data.

2.1 3-Letter Axis Notation

Letter	Meaning
R / L	right / left
F / B	forward / backward
U / D	up / down

Common source systems:

- `RFU` — already correct (GTA V, Unreal Engine)
  - UE5's world axes (X=forward, Y=right, Z=up, left-handed) are not the same as the spec's RFU (X=right, Y=forward, Z=up, right-handed) — the X and Y are swapped and the handedness is opposite.
- `RUB` — OpenGL/Three.js native: swap Y↔Z
- `FLU` — ROS: remap X→Y, Y→−X

- `RDF` — OpenCV: remap Y→−Z, Z→Y

2.2 Converting Camera Columns to OpenGL Convention

- Compute **forward** from the source matrix (may require Euler decomposition)
- **right** = normalize(forward × world_up) — always horizontal
- **up** = normalize(right × forward) — gravity-aligned
- **backward** = −forward
- **C2W** = [right | up | backward | position]
- Verify `det(R) = +1`

> ⚠ **Warning:** Do not use similarity transforms ($P \times M \times P^{-1}$) for axis swaps with sign changes — they can flip pitch/roll signs. Remap each vector individually instead.

2.3 (GTA V) Conversion Notes

- **Affected data:** r2 bucket
- **Source world:** `RFU` (X=east, Y=north, Z=up) — same as canonical
- **Source C2W:** Euler-encoded R_Y(yaw) × R_X(pitch), M10 = 0 always
- Heading: `atan2(−M20, M00)`
  - Pitch: `atan2(−M12, M11)`
- **Source intrinsics:** `FOV_Deg` (vertical) + `FOV_Axis` column
- **Source inputs:** comma-separated keys, "NONE" for empty
- **Conversion:** decompose Euler → reconstruct via cross products → compute f_x/f_Y from FOV + resolution → pipe-delimit inputs

2.4 (RDR2) Conversion Notes

- **Affected data:** r2 bucket
- **Source world:** Z-up (position Z has smallest range)
- **Source C2W:** DirectX view matrix (row-vector, left-handed, det = −1) — **do not use directly**
- **Source Euler:** `rot_euler = [pitch, roll, yaw]` in degrees
- Yaw reference: +Y axis (north)
  - forward = (−sin(yaw)×cos(pitch), cos(yaw)×cos(pitch), sin(pitch))
- **Source intrinsics:** projection matrix with `proj[0][0]` and `proj[5]`
  - Use f_Y = `proj[5] × height/2`, $f_x = f_Y$
- **Source inputs:** event-based JSONL (keyboard.jsonl, mouse.jsonl) at variable rate
- **Camera data:** optional — some sessions (e.g. game-data bucket) lack camera.jsonl
- **Frame rate:** camera at ~120fps, video at 30fps — resample to video fps
- **Conversion:** Euler → cross products → OpenGL C2W. Resample inputs to frame rate. Pipe-delimit keys.

2.5 Cyberpunk Notes

- **Affected data:** r2 bucket
- ⚠️ ***Sync issue with ~109.9 hrs of Cyberpunk delivered on and prior to 04/06/26. (Wall time was fetched from the game clock not global clock).***

2.6 Hogwarts Legacy Notes

- **Affected bucket:** r2 bucket
- Hogwarts Legacy data needs to go through the §4.2 column-reconstruction process before delivery — decompose the UE5 actor-space matrix, re-express the axes in right-handed world space, and assemble [right | up | backward | position]